

Basic Laws of Robotics and Feedback Model

Corrado Santoro

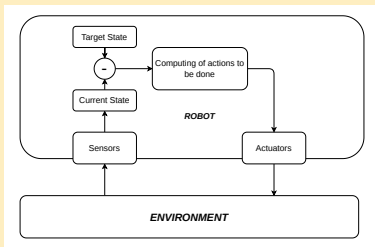
Dipartimento di Matematica e Informatica
corrado.santoro@unict.it



Corso di "Introduction to Robotics and AI"

Back to Robot's Working Model

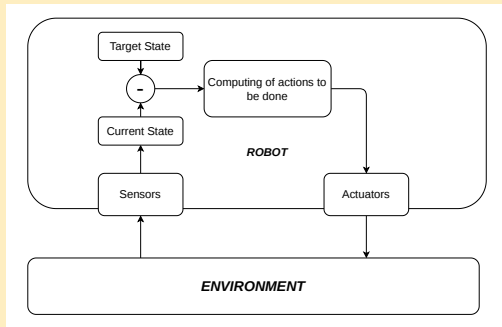
Behaviour Model



The Five Axioms of Robotics

Axioms

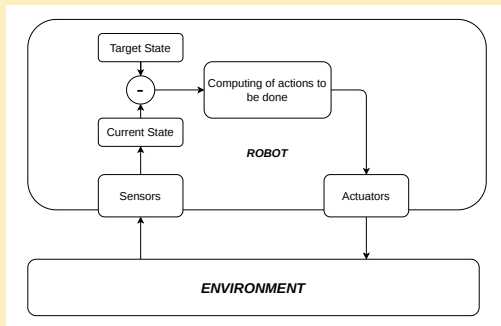
Axiom 1: Robots trust sensors



- The robot acts on the basis of sensor's data
- If sensor's data are **wrong** (i.e. do not reflect the reality) the robot behaves bad
- In the same way, when we “wrongly perceive” an information, we make errors!!

Axioms

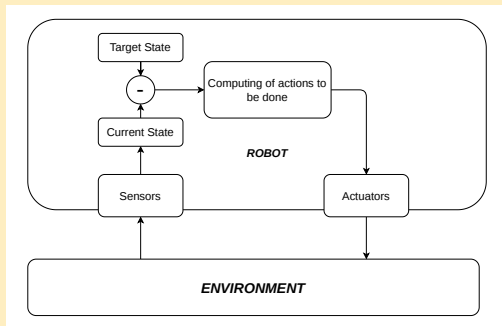
Axiom 2: “Things” that are not perceived do not exist



- There is a big difference between what we perceive and what a robot perceives
- We must remind that our senses **are not** robot's senses

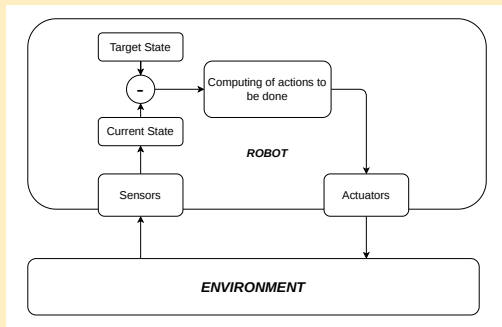
Axioms

Axiom 3: The Environment must be known



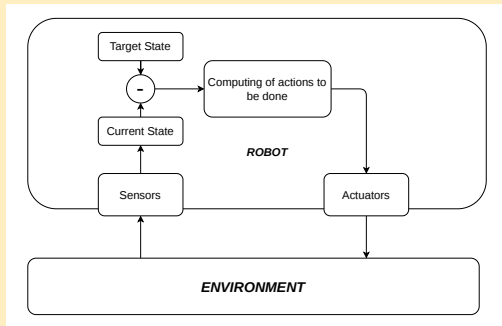
- The “behaviour” of the robot (i.e. the software) must know the **structure of the environment** and of itself, otherwise it cannot “decide” and operate
- A “good” **representation** of the environment must be encoded
- If the environment **cannot be known a-priori**, a proper “**mapping strategy**” must be employed (by means of the use of sensors)

Axiom 4: The Environment is dynamic and unpredictable



- We cannot make any assumption about the environment, **anything can happen!**
- Physical environments are dynamic and unpredictable
- A robot must be able to face **unpredictable events** and respond accordingly in order to restore the “target state”

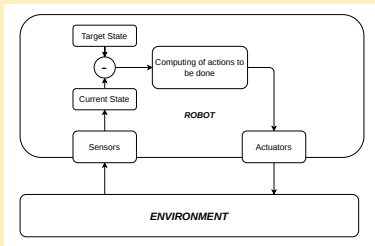
Axiom 5: Actions fail



- We cannot assume that “making an action” implies that the **action succeeds**!
- **Things may go wrong**, and the robot must be able to detect failures and respond accordingly

Robotics by Example

Behaviour Model



The Cruise (Speed) Control

Speed Control

Human Car Speed Control

What is our behaviour if we want to keep our speed constant?

- 1 We decide (in our mind) our **target speed** (let's call it v_t)
- 2 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 3 If v_c is **quite equal to** v_t , we do not move our gas pedal
- 4 If v_c is **greater than** v_t ($v_c > v_t$), we **decrease the pressure to the gas pedal**
- 5 If v_c is **less than** v_t ($v_c < v_t$), we **increase the pressure to the gas pedal**

Axioms

- 1 Robots trust sensors
- 2 "Things" that are not perceived do not exist
- 3 The Environment must be known
- 4 The Environment is dynamic and unpredictable
- 5 Actions fail

Speed Control

Human Car Speed Control

What is our behaviour if we want to keep our speed constant?

- 1 We decide (in our mind) our **target speed** (let's call it v_t)
- 2 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 3 If v_c is **quite equal to** v_t , we do not move our gas pedal
- 4 If v_c is **greater than** v_t ($v_c > v_t$), we **decrease the pressure to the gas pedal**
- 5 If v_c is **less than** v_t ($v_c < v_t$), we **increase the pressure to the gas pedal**

Gas Pedal Pressure

- 1 How do we decide the **amount** of pressure increment/decrement for points 4 and 5?
- 2 We know that when we apply a (sudden) pressure change, car's speed (v_c) changes gradually
- 3 In other words, the car (system) has a certain **latency**
- 4 Latency is a **physical effect** of **any real system**

Speed Control

Human Car Speed Control

What is our behaviour if we want to keep our speed constant?

- 1 We decide (in our mind) our **target speed** (let's call it v_t)
- 2 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 3 If v_c is **quite equal to** v_t , we do not move our gas pedal
- 4 If v_c is **greater than** v_t ($v_c > v_t$), we **decrease the pressure to the gas pedal**
- 5 If v_c is **less than** v_t ($v_c < v_t$), we **increase the pressure to the gas pedal**

Gas Pedal Pressure

- 1 The car (system) has a certain **latency**
- 2 We **know by experience** this latency
- 3 So we can **decide** the amount of increment/decrement
- 4 If our **increment** is **low**, the car accelerates slow and thus **achieves target speed slowly**
- 5 If our **increment** is **high**, the car accelerates fast and thus **achieves target speed fastly**, but ...
- 6 ... **target speed could be overcome** thus requiring pressure decrement

Speed Control

From Human Behaviour to Math

What is our behaviour if we want to keep our speed constant?

- 1 Let's call **GAS** the **percentage of pressure** applied to the gas pedal
- 2 We decide (in our mind) our **target speed** (let's call it v_t)
- 3 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 4 If v_c is **quite equal** to v_t , we do not move our gas pedal
- 5 If v_c is **greater than** v_t ($v_c > v_t$), we **decrease GAS**
- 6 If v_c is **less than** v_t ($v_c < v_t$), we **increase GAS**
- 7 Apply **GAS** to the pedal

Speed Control

From Human Behaviour to Math—Introducing *GAS*

What is our behaviour if we want to keep our speed constant?

- 1 Let's call *GAS* the **percentage of pressure** applied to the gas pedal
- 2 We decide (in our mind) our **target speed** (let's call it v_t)
- 3 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 4 If v_c is **quite equal to** v_t , we do not move our gas pedal
- 5 If v_c is **greater than** v_t ($v_c > v_t$), decrease *GAS*, $GAS = GAS - K$
- 6 If v_c is **less than** v_t ($v_c < v_t$), increase *GAS*, $GAS = GAS + K$
- 7 Apply *GAS* to the pedal

K is a constant (decided by us) that we use to increment/decrement our pressure

Speed Control

From Human Behaviour to Math—Introducing *error*

What is our behaviour if we want to keep our speed constant?

- 1 Let's call *GAS* the **percentage of pressure** applied to the gas pedal
- 2 We decide (in our mind) our **target speed** (let's call it v_t)
- 3 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 4 Let's compute the **speed error** $\Rightarrow error = v_t - v_c$
- 5 If $error \simeq 0$, do not change *GAS*
- 6 If $error < 0$, $GAS = GAS - K$
- 7 If $error > 0$, $GAS = GAS + K$
- 8 Apply *GAS* to the pedal

Speed Control

From Human Behaviour to Math—Modulating K

What is our behaviour if we want to keep our speed constant?

- 1 Let's call **GAS** the **percentage of pressure** applied to the gas pedal
- 2 We decide (in our mind) our **target speed** (let's call it v_t)
- 3 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 4 Let's compute the **speed error** $\Rightarrow error = v_t - v_c$
- 5 If **error** $\simeq 0$, do not change GAS
- 6 If **error** < 0 , **GAS** = **GAS** - K
- 7 If **error** > 0 , **GAS** = **GAS** + K
- 8 Apply **GAS** to the pedal

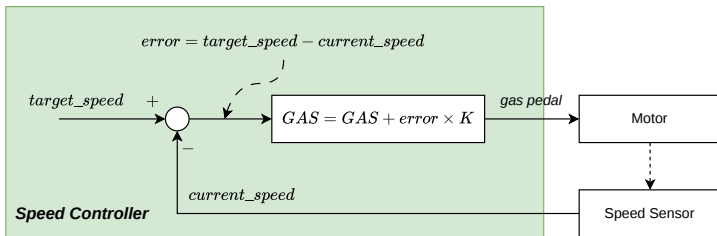
- In reality, we **modulate** K on the basis of the error
- We do not use **the same** K if the error is **low** or **high**
- When **error is low** we apply a **small increment/decrement**
- When **error is high** we can apply a **high increment/decrement**
- Indeed, we use an **increment/decrement** that is **proportional to the error**

Speed Control

From Human Behaviour to Math—Modulating incr/decr with *error*

What is our behaviour if we want to keep our speed constant?

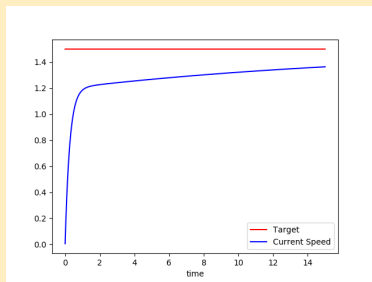
- 1 Let's call **GAS** the **percentage of pressure** applied to the gas pedal
- 2 We decide (in our mind) our **target speed** (let's call it v_t)
- 3 We periodically read, from car's panel, the **car's current speed** (let's call it v_c)
- 4 Let's compute the **speed error** $\Rightarrow error = v_t - v_c$
- 5 Let's update **GAS** $\Rightarrow GAS = GAS + error \times K$
- 6 Apply **GAS** to the pedal



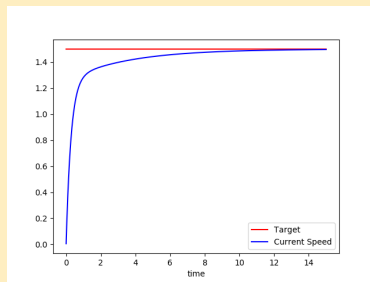
Speed Control

Effects of K

Too low



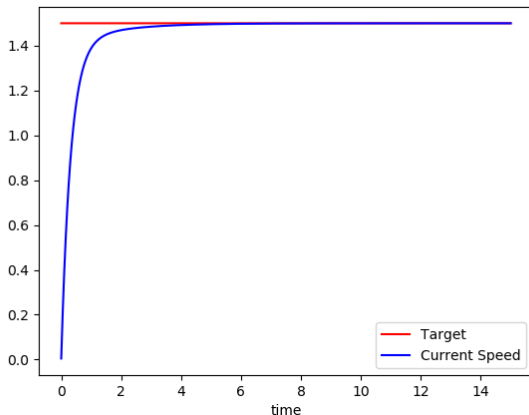
Low



Speed Control

Effects of K

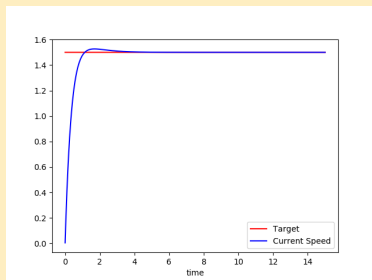
Fair



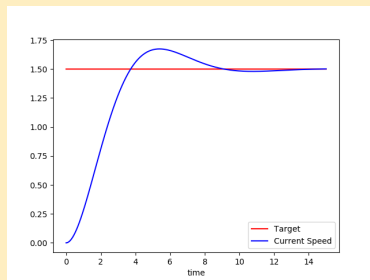
Speed Control

Effects of K

High



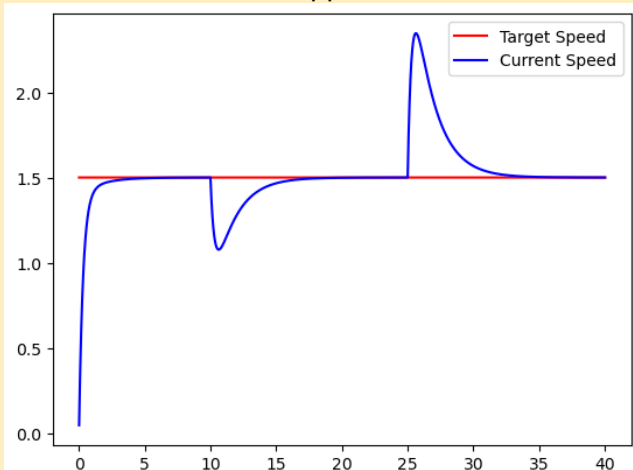
Too High



Speed Control

Effects of environment changes

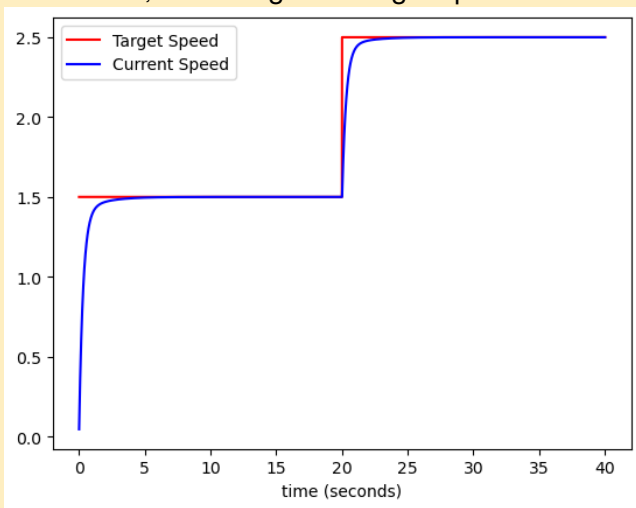
- After 10 seconds, a climb appears
- After 25 seconds, a descent appears



Speed Control

Effects of environment changes

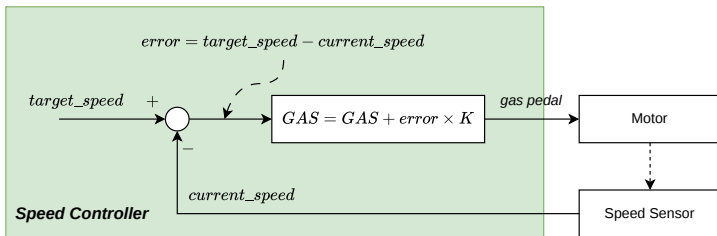
- After 20 seconds, we change the target speed



Speed Control

The Immediate Reaction and Long-time Reaction

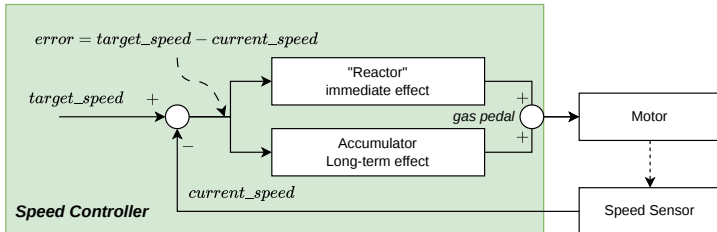
- In practice, to avoid **oscillation**, when we **in proximity** of the target speed, we must use a **low K**
- But if we are **far** from the target speed, a **low K** will imply **too much time** to approach the target speed
- So, if we are **far** from the target speed, we should use a **high K**
- Do we have to use a **variable K** ? Not so convenient...



Speed Control

The Immediate Reaction and Long-time Reaction

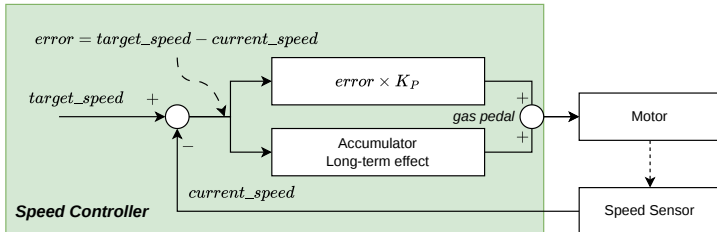
- We add another **strategy block**, able to **react immediately**, when the **error is high**, thus making the system to **approach** fast to the target
- So, now that **error became low**, we can let the **accumulator** do its job, and use a **low K** in order to **reduce the error** avoiding oscillations.



Speed Control

The Immediate Reaction and Long-time Reaction

- This additional **strategy block** can be very simple, i.e. something that generates a **gas pedal pressure** as **proportional to error**, weighted by another **constant K_P**
- It cannot work alone (without the accumulator), since when **error = 0**, it would generate a **gas pedal pressure which is 0** (thus stopping the car!!!)

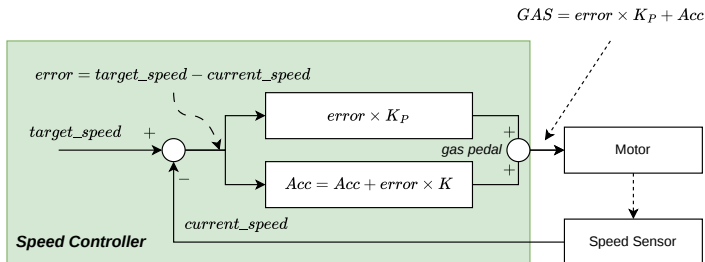


Speed Control

The Immediate Reaction and Long-time Reaction

- So let's change the variable used by the accumulator, instead of *GAS*, we introduce *Acc* ...
- because now the effective *gas pedal pressure GAS* is now given by the contribution of both blocks:

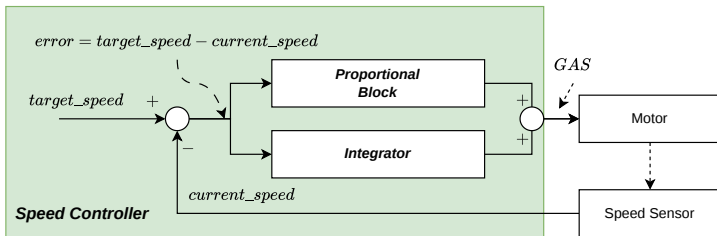
$$GAS = error \times K_P + Acc$$



Speed Control

The “PI” Control

- From the control theory point of view, the upper block (i.e. what we called “reactor”) is called **Proportional block**, and its **action** is governed by the constant K_P
- The accumulator is called **Integral block** or **Integrator**, and its action is governed by the constant indeed called K_I (rather than simply K)
- The overall control strategy is called **Proportional-Integral** or simply **“PI”**



Speed Control

The Speed Controller as a “Black Box”

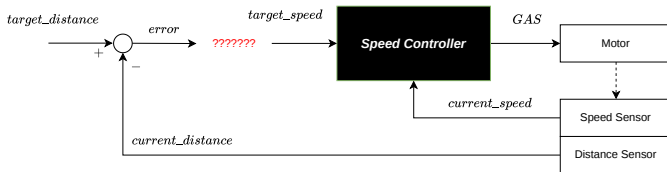
- Now that we solved the problem of letting our car drive at the speed we desire, we can “close” the speed controller in a **block box**, hiding the details, because we are no more interested in them
- It is now something that, by giving as input our *target_speed* and the *current_speed* read by the sensor, is able to **ensure** that the car drives at our *target_speed*
- May we use it for other automation purposes ?

Position Control

Position Control

Position Control

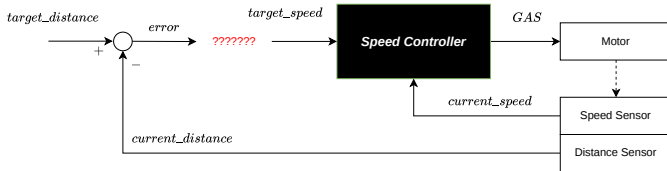
- Now let's suppose we want to make the car **travel a certain distance** (on a straight line, we do not mind steering) from a certain starting point
- Let's suppose we have **sensor** able to measure the **distance traveled** by the car from the starting point. We have two variables:
 - target_distance**, our desire, fixed by us
 - current_distance**, measured by the sensor
- And we can compute, also in this case, an **error**:
$$\text{error} = \text{target_distance} - \text{current_distance}$$
- Can we think to a **“strategy”** that uses the distance error and the speed controller to make the car cover the desired distance?



Position Control

Position Control

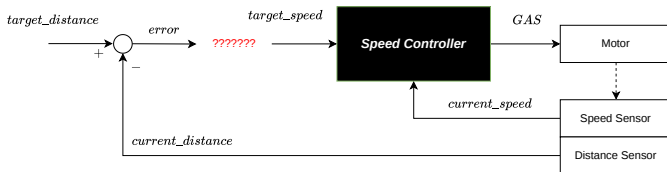
- Practically, we drive at **certain (constant) speed** when we are **far** from the target
- We **reduce the speed** as soon as we **approach** the target
- We **stop** $speed = 0$ when we **reached the target**



Position Control

Position Control

- 1 We drive at **certain (constant) speed** when we the **error is high**
- 2 We **reduce the speed** according to the **error reduction**
- 3 We **stop speed = 0** when **error = 0**



Position Control

Position Control

- 1 We drive at **certain (constant) speed** when we the **error is high**
- 2 We **reduce the speed** according to the **error reduction**
- 3 We **stop speed = 0** when **error = 0**

Position Proportional Control

- What about a **proportional strategy** also in this case?

$$\text{target_speed} = (\text{target_position} - \text{current_position}) \times K_P$$

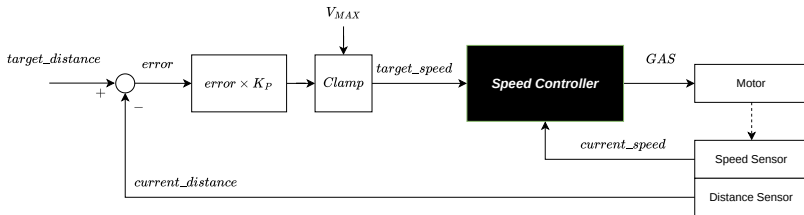
$$\text{target_speed} = \text{error} \times K_P$$

- It meets points 2 and 3 of our “practical strategy”!!
- If we **avoid** that the **target_speed** computed **overcomes** a certain V_{MAX} (decided by us), we meet also point 1

Position Control

Position Proportional Control

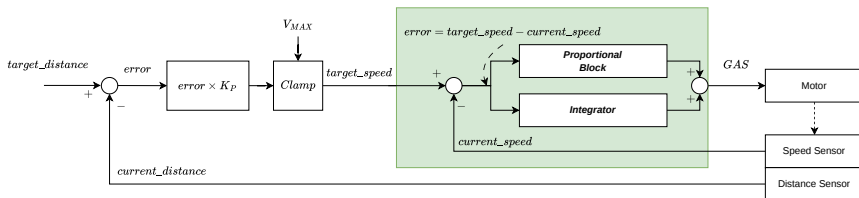
- We employ a simple **proportional block**, with a suitable K_P , ...
- ...followed by a “**clamp**” block that is able to ensure that our speed **never overcomes a certain V_{MAX}** (decided by us)
- The speed computed is then **guaranteed** by the **speed controller**



Position Control

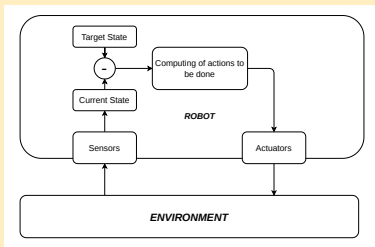
Position + Speed Control Revealed

- Now let's re-open the **speed control** black box
- As you can see, we have a **double feedback** schema
- Multiple (cascading) feedbacks are very usual in robotics, and, according to the structure of the robot and the number of actuators, we can have **tens** or even **hundreds** of feedback loops, interconnected and working together



Robotics by Example

Behaviour Model



Factory Robots

(videos)

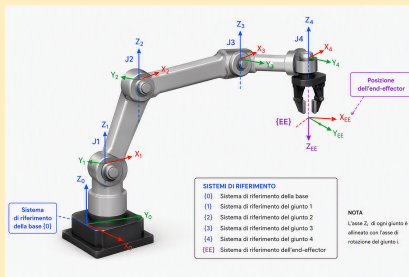
Factory Robots

Feedback or not?

- Factory robots always execute a **prefixed** sequence of movements
- They do not sense e.g. where is the car, because the car will arrive **always in that same position**
- So the environment is designed **ad-hoc** to make robot's actions always succeed
- If something goes wrong, a safety system **stops everything**, i.e. the overall production chain
- **Does it mean that the feedback mechanism is not present?**

Robotic Arms

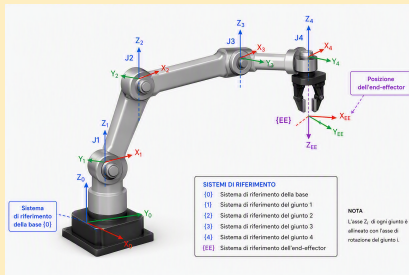
How is the movement of a robotic arm managed?



- A **robotic arm**, also called **manipulator**, is made of a **serial** chain of:
 - **Rotating Joints**, driven by proper **electric motors**
 - **Links**, made by mechanical parts that connect joints
- At the end of the chain, a **tool** is present, e.g. a gripper, a spray, a solder, a sucker, that is called **end-effector**

Robotic Arms

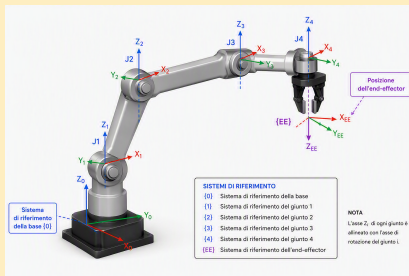
How is the movement of a robotic arm managed?



- Movement is performed by making the **end-effector** follow a certain **trajectory** in the working space
- The trajectory is made of a **set of points** that are expressed in **cartesian coordinates** (x, y, z) (w.r.t. robot's base)
- We can only **drive joints**, in terms of **amount of rotation** of each motor

Robotic Arms

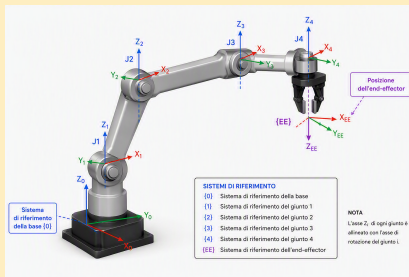
Inverse Kinematics



- We can only **drive joints**, in terms of **amount of rotation** of each motor
- The problem is: given a **target point** (x_t, y_t, z_t) what are the **rotations** of each joint to let the **end-effector** reach the target?
- This problem is called **inverse kinematics** and its solution **is not unique**

Robotic Arms

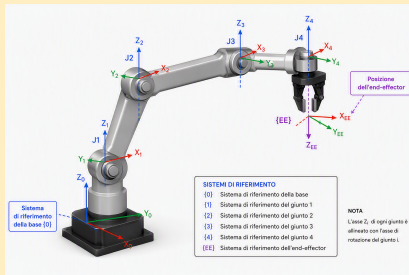
Direct Kinematics



- We can only **drive joints**, in terms of **amount of rotation** of each motor
- The problem is: given the **rotations** of each joint ($j_1, j_2, \dots, j_n$), what is the **position** in the space (x_e, y_e, z_e) of the **end-effector**?
- This problem is called **direct kinematics** and its solution is **unique**

Robotic Arms

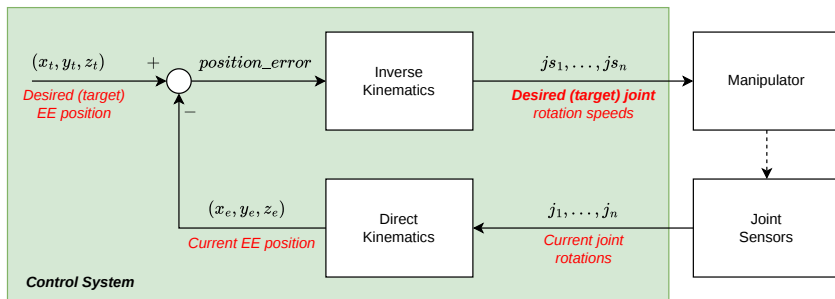
Sensors in Robotic Arms



- We can only **drive joints**, in terms of **amount of rotation** of each motor
- **Joints** (i.e. motors) have proper **sensor** that are able to measure the **amount of rotation** (in degrees) and the **rotation speed**
- We cannot **directly measure** the position in the space of the **end-effector**
- We cannot **directly drive** the position in the space of the **end-effector**

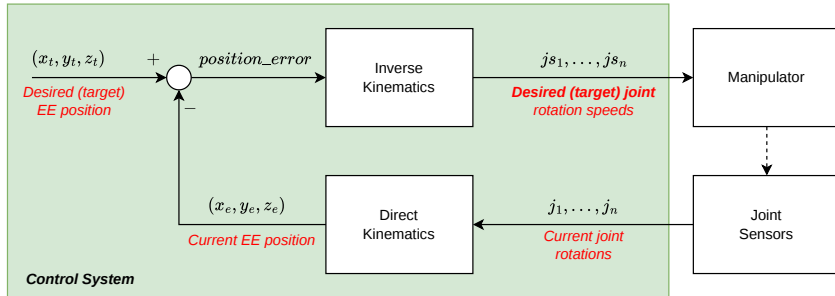
Robotic Arms

Here is the feedback!!



Robotic Arms

Is the problem solved?



- **Warning!!** The outputs of Inverse Kinematics are the **desired speeds** of each joint
- **How can we assure that each joint rotates at that speed??**

Robotic Arms

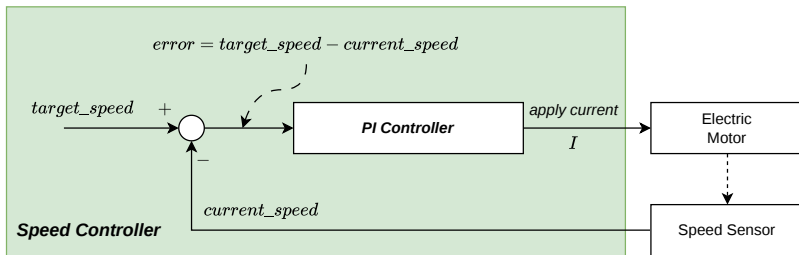
Electric Motors



- Electric motors work using **electric current**
- By **modulating** the electric current, we can **vary the speed** (something like the degree of pressure of the gas pedal in cars)
- But **unless we put a sensor inside**, we **cannot know the real motor speed**
- To assure a certain **desired/target speed**, we **must employ** exactly the **same schema** used in the **car's cruise control**

Robotic Arms

Electric Motors

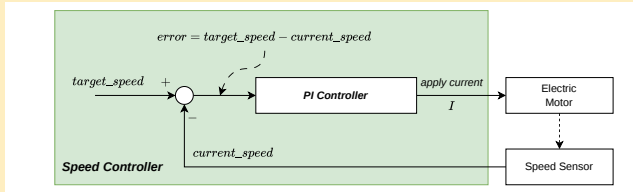
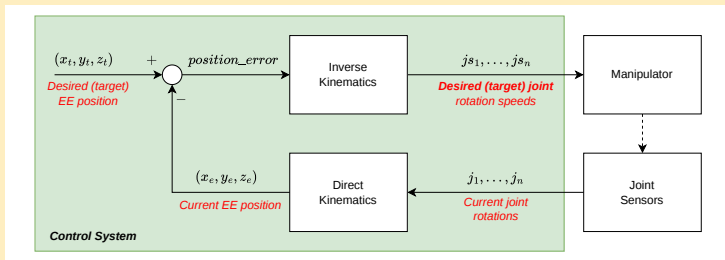


- Instead of using **GAS pedal**, we consider the **intensity of the electric current "I"** to be given to motor
- Details change (since the system is different) but the **control strategy/schema** remains the same

Robotic Arms

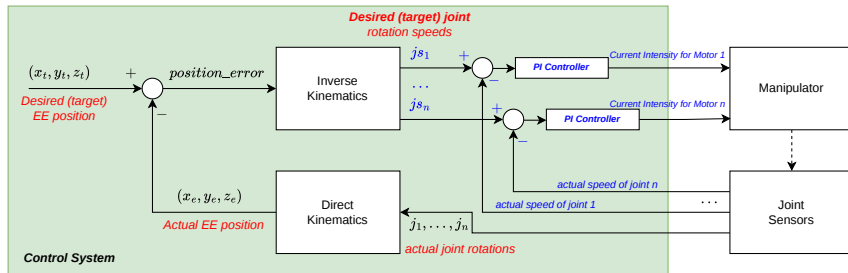
Back to Manipulator: the “Chinese Boxes”

We must **combine both schemas**



Robotic Arms

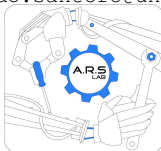
The Final Control Schema (“Chinese Boxes”)



Basic Laws of Robotics and Feedback Model

Corrado Santoro

Dipartimento di Matematica e Informatica
`corrado.santoro@unict.it`



Corso di “Introduction to Robotics and AI”